ARQUITECTURA DEL SOFTWARE

2025-26

Jose Emilio Labra Gayo
Pablo González
Celia Melendi
Diego Martín

Escuela de Ingeniería Informática

Universidad de Oviedo

Laboratorio 6

TDD: Test-driven development
Cobertura de código(SonarCloud)
Integración continua (GitHub Actions)
Herramientas para el análisis estático (SonarCloud)

TDD - Introducción

- Proceso de desarrollo de código donde se propone comenzar definiendo casos de prueba y luego el Código
  - Respuesta al desarrollo donde los test se dejaban para fase final
  - Técnica propuesta por Kent Beck
- Desarrollo Rojo/Verde
  - Al comienzo, las pruebas fallan (no hay código) ⇒ Rojo
  - A medida que se añade código las pruebas van pasando ⇒ Verde

TDD - Fases

1. Añadir un caso de prueba
2. Ejecutar casos de prueba
   - Si algún caso falla
3. Re-escribir el código
4. Ejecutar todas las pruebas
5. Refactorización del código

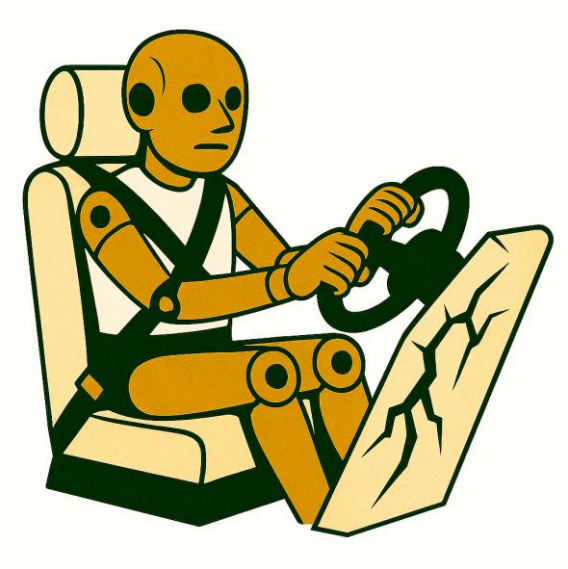
TDD - Características

Código sencillo que satisface las necesidades del cliente
Obtenemos código sencillo + Batería de pruebas
Ayuda a centrarse en qué se quiere desarrollar

Creación de Pruebas

- Principios FIRST
  - F - Fast
    - La ejecución de pruebas debe ser rápida
  - I - Independent:
    - Los casos de prueba son independientes entre sí
  - R - Repeatable:
    - Tras ejecutarlos N veces, el resultado debe ser el mismo
  - S - Self-checking
    - Se puede comprobar si se cumplen automáticamente, sin intervención humana
  - T - Timely
    - Pruebas escritos al mismo (o antes) tiempo que código

Dobles de pruebas



- Objetos *Dummy:*
  - Se pasan pero no se utilizan
- Objetos *falsos (fake):*
  - Tienen implementación parcial
- *Stubs:*
  - Respuestas precocinadas a ciertas preguntas
- *Espías: stubs* que pueden registrar cierta información para depuración
- *Mocks: simulan comportamiento de otros objetos*
  - Programados con ciertas expectativas sobre qué tipo de llamadas deben recibir
- *Fixtures*. Elementos fijos de soporte a las pruebas
  - *Ej. Bases de datos con ciertas entradas, determinados ficheros, etc.*

Pruebas en Rust

- Pruebas unitarias pueden incluirse en cada fichero
  - Se marcan con #[cfg(test)] en un módulo tests
- Pruebas de integración
  - Se incluyen en carpeta `tests`
- Se ejecutan mediante: `cargo test`
- Se puede ejecutar una prueba concreta mediante:
  - `cargo test nombre`

```
#[cfg(test)]
mod tests {
    use super::*;
    use crate::{Movement, PlayerId};


 #[test]
    fn test_random_bot_name() {
        let bot = RandomBot;
        assert_eq!(bot.name(), "random_bot");
    }
. . .
```

Pruebas en React: vitest

- Framework de pruebas para vite
  - vite = servidor desarrollo para React, vue.js, Svelte
  - Monitoriza ficheros mientras son editados
- Soporte para Typescript, JSX
  - Compatible con React, vue.js
- Descripciones de pruebas
  - Vocabulario:
    - describe: permite agrupar pruebas
    - it: define un caso de prueba concreto
    - expect: declarar aserciones

TDD – Test de ejemplo (servicio web)

- Prueba de servicio web básico

```
import { describe, it, expect, afterEach, vi } from 'vitest'
import request from 'supertest'
import app from '../users-service.js'

describe('POST /createuser', () => {
    it('returns a greeting message for the provided username', async () => {
        const res = await request(app)
            .post('/createuser')
            .send({ username: 'Pablo' })
            .set('Accept', 'application/json')
        expect(res.status).toBe(200)
        expect(res.body).toHaveProperty('message')
        expect(res.body.message).toMatch(/Hello Pablo! Welcome to the course!/i)
    })
})
```

TDD – Test de ejemplo (React)

Comprobar que RegisterForm funciona bien:

A veces tenemos que moquear parte de la prueba.
Si no mockeamos el api, nuestro test depende de UserService de los resultados de la *rest-api*.
Como se trata de test unitarios debemos eliminar esta dependencia.

```
test('submits username and displays response', async () => {
  const user = userEvent.setup()

  // Mock fetch to resolve automatically
  global.fetch = vi.fn().mockResolvedValueOnce({
    ok: true,
    json: async () => ({ message: 'Hello Pablo! Welcome to the course!' }),
  } as Response)

  render(<RegisterForm />)

  await waitFor(async () => {
    await user.type(screen.getByLabelText(/whats your name\?/i), 'Pablo')
    await user.click(screen.getByRole('button', { name: /lets go!/i }))
    expect(
      screen.getByText(/hello pablo! welcome to the course!/i)
    ).toBeInTheDocument()
  })
})
```

Cobertura de código - Code Coverage

Cobertura de código: Medida que nos indicala proporción de líneas de c código que son probadas en alguno de nuestros test.

SonarCloud: Herramienta que incluye la cobertura de código como una métrica más en la evaluación del código.

Sonar Cloud recoge los datos de los resultados del lanzamiento de los test, recogiendo los siguientes valores:

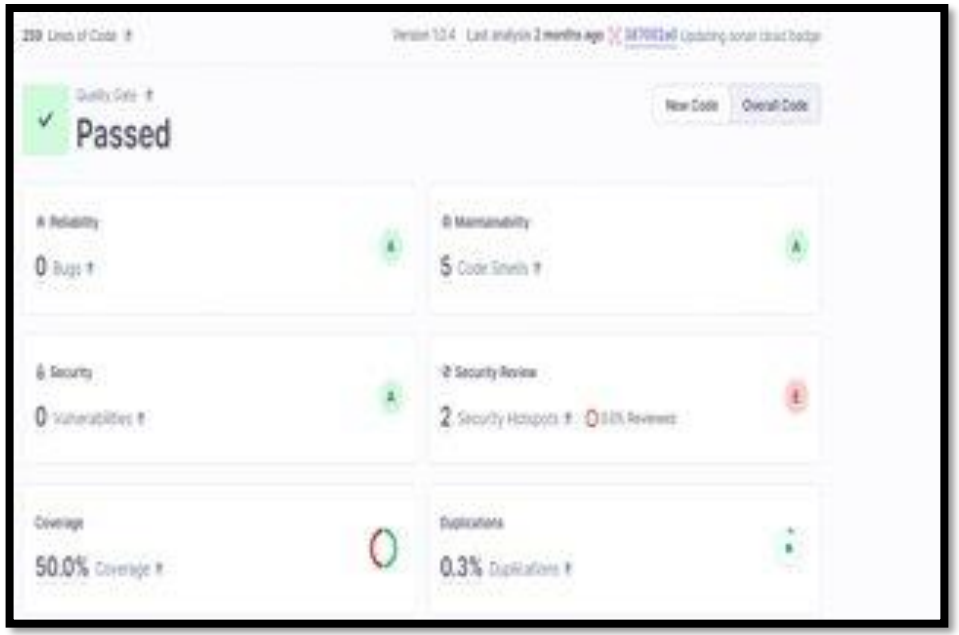
- LC = Líneas cubiertas (lines_to_cover - uncovered_lines)
- EL = Número total de líneas ejecutables (lines_to_cover)

SonarCloud

La ratio de cobertura se calcula con la fórmula: LC/EL

Tras ejecución de tests, se genera fichero para su posterior análisis:

https://sonarcloud.io/summary/overall?id=Arquisoft_yovi_???



Integración Continua - Definición

Práctica de desarrollo que promueve la integración del código de forma frecuente

El lanzamiento del proceso de integración continua es ejecutado cuando se cumple alguna condición

- Cada vez que se genera un *push*, un *pull request*, etc.

Integración Continua - Mejoras

Detecta y resuelve problemas de una manera continua
Siempre hay una versión disponible
Ejecución automática de casos de prueba
Ejecutar pruebas en un entorno externo
Facilita despliegue automático
Permite monitorizar la calidad de código

Integración Continua - sistemas

- Jenkins
- Pipeline
- Hudson
- Apache Continuun
- Travis
- **GitHub Actions**

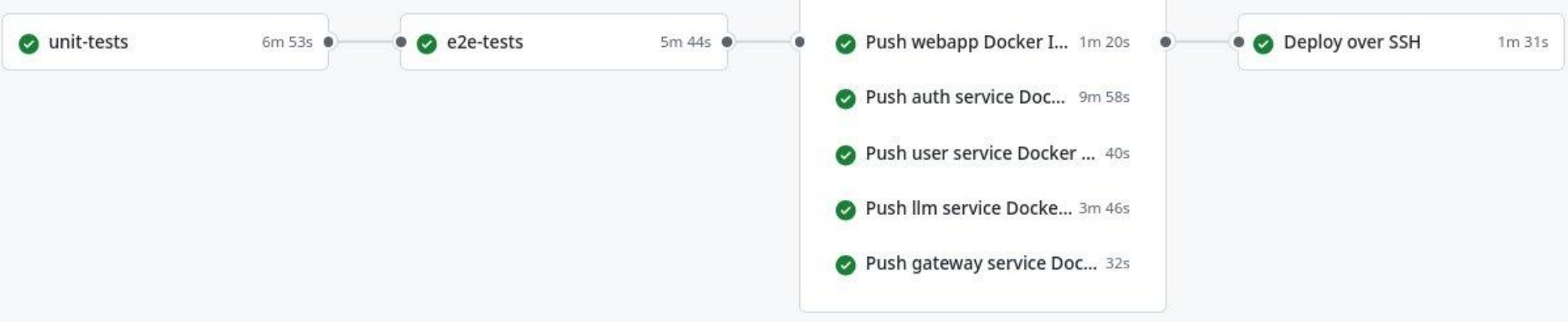
GitHub Actions

- Permite gestionar la integración continua sobre los proyectos de los repositorios en GitHub
  - Gratis para proyectos de código Abierto
- Configuración se realiza en ficheros .yml en: **`.github/workflows`**

GitHub Actions

- Contenido .yml :
  - Condiciones que lanzan el proceso (On)
  - Lista de tareas (Jobs)
    - Cada tarea ejecutada en su propio entorno.
  - Especificación para cada tarea
    - checkout, install dependencies, build y test

```yaml
jobs:
  unit-tests:
    runs-on: ubuntu-latest
    steps:
    - uses: actions/checkout@v4
    - uses: actions/setup-node@v4
      with:
        node-version: 22
    - run: npm --prefix users/authservice ci
    - run: npm --prefix users/userservice ci
    - run: npm --prefix llmservice ci
    - run: npm --prefix gatewayservice ci
    - run: npm --prefix webapp ci
    - run: npm --prefix users/authservice test -- --coverage
    - run: npm --prefix users/userservice test -- --coverage
    - run: npm --prefix llmservice test -- --coverage
    - run: npm --prefix gatewayservice test -- --coverage
    - run: npm --prefix webapp test -- --coverage
    - name: Analyze with SonarQube
      uses: SonarSource/sonarqube-scan-action@master
      env:
        SONAR_TOKEN: ${{ secrets.SONAR_TOKEN }}
```



GitHub Actions

- Cada tarea debe tener un propósito específico
  - Probar una parte de la app, desplegar, etc.
- Se puede usar para automatizar otras partes del repositorio.
  - Ejemplo: responder automáticamente cuando se crea un nuevo issue.

GitHub Actions

- Tambien tenemos jobs para crear imágenes de Docker y publicarlas
- Véase documentación para más configuraciones

```yaml
docker-push-webapp:
  name: Push webapp Docker Image to GitHub Packages
  runs-on: ubuntu-latest
  permissions:
    contents: read
    packages: write
  needs: [e2e-tests]
  steps:
  - uses: actions/checkout@v4
  - name: Publish to Registry
    uses: elgohr/Publish-Docker-Github-Action@v5
    env:
      API_URI: http://${{ secrets.DEPLOY_HOST }}:8000
    with:
        name: arquisoft/wichat_0/webapp
        username: ${{ github.actor }}
        password: ${{ secrets.GITHUB_TOKEN }}
        registry: ghcr.io
        workdir: webapp
        buildargs: API_URI
```

Análisis estático del código

Analiza el código sin compilarlo

Detecta bugs, code smells, vulnerabilidades del sistema, etc

Útil para medir la calidad del código.

Se puede bloquear la subida de código que no cumpla con ciertas características de calidad

Análisis estático en SonarCloud

sonarcloud

SonarCloud contiene herramientas para análisis estático del código

Necesita:

- Servidor de Git como GitHub
- Acceso al repositorio
- Un lenguaje soportado

Dos clases de configuración de los análisis:

- **Automated Analysis** (Default). Cobertura de código no disponible. Scanner del código en servidor sonar.
- **CI-based analysis**. Sonar scanner ejecutado externamente. Los report son enviados a SonarCloud.

Configuración de Sonar

- Fichero sonar-project.properties

```
sonar.organization=arquisoft
sonar.host.url=https://sonarcloud.io
sonar.projectKey=Arquisoft_yovi_0
sonar.projectName=yovi_0

sonar.sources=webapp/src,users,gamey/src
sonar.tests=webapp/src,users/__tests__,gamey/src
sonar.test.inclusions=**/*.test.tsx,**/*.test.js
sonar.javascript.lcov.reportPaths=webapp/coverage/lcov.info,users/coverage/lcov.info
sonar.rust.lcov.reportPaths=gamey/lcov.info
```

Sonarlint

sonarlint

Análisis estático desde el propio IDE (disponible en los más populares ej. IntelliJ, Visual Code, Visal Studio, Eclipse…)

Provee de análisis estático de forma local (antes de subirlo al repositorio), se ejecuta:

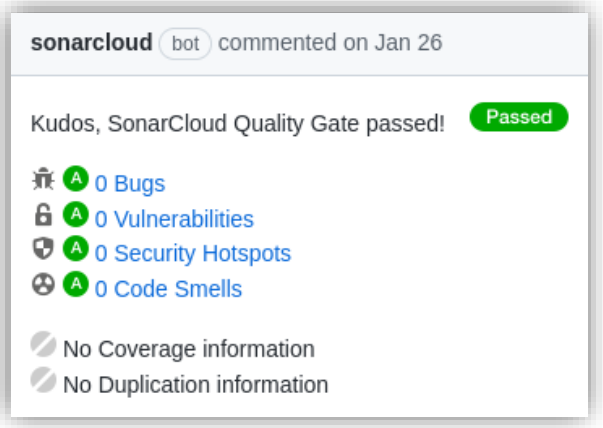
- De forma manual
- Automáticamente sobre los archivos modificados, antes de enviarlos al repositorio.

Más información, IDEs-lenguajes soportados y cómo instalarlo en la página oficial

SonarCloud – yovi_x

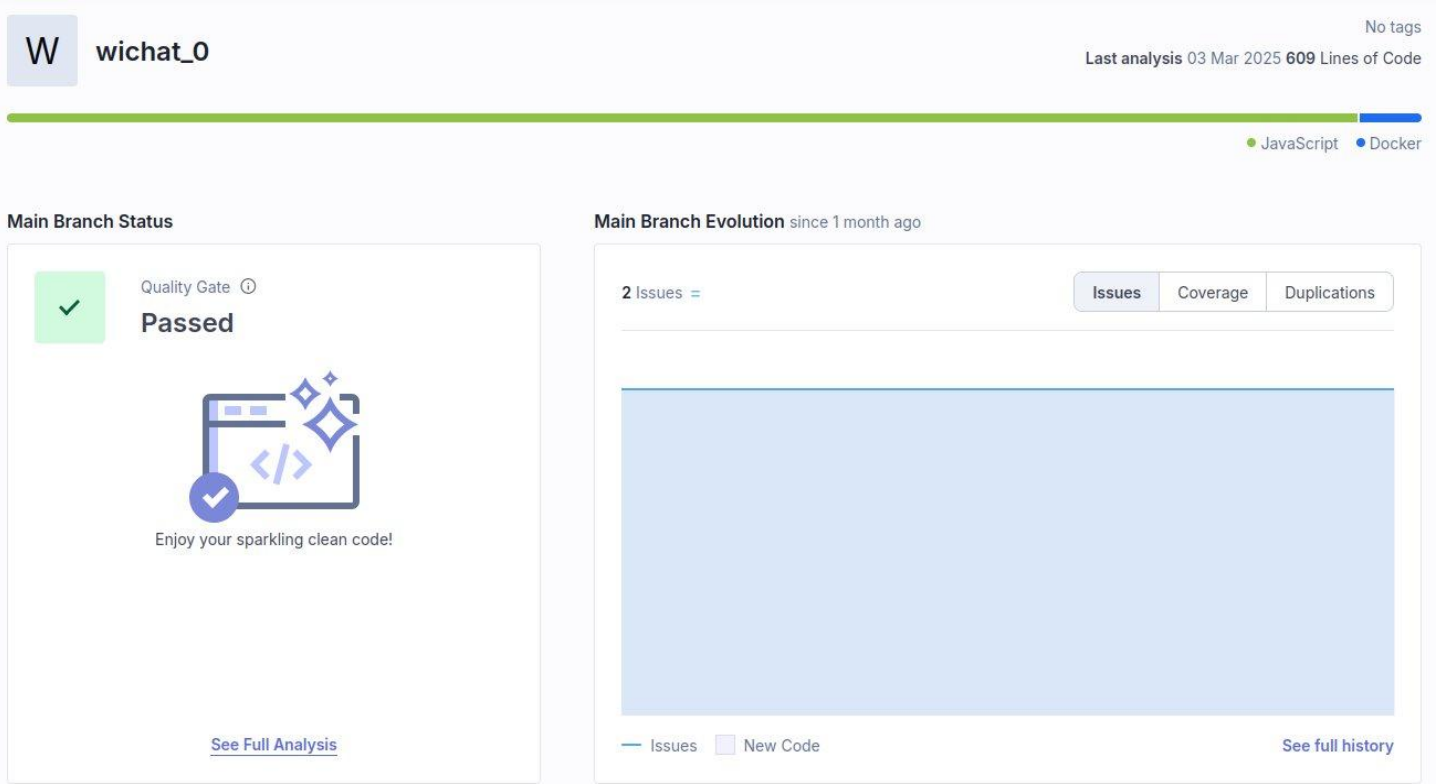
Cuando los cambios son enviados al repositorio (ejemplo: una nueva pull request)

Tenemos información acerca de la calidad de código del pull request que estamos mergeando.

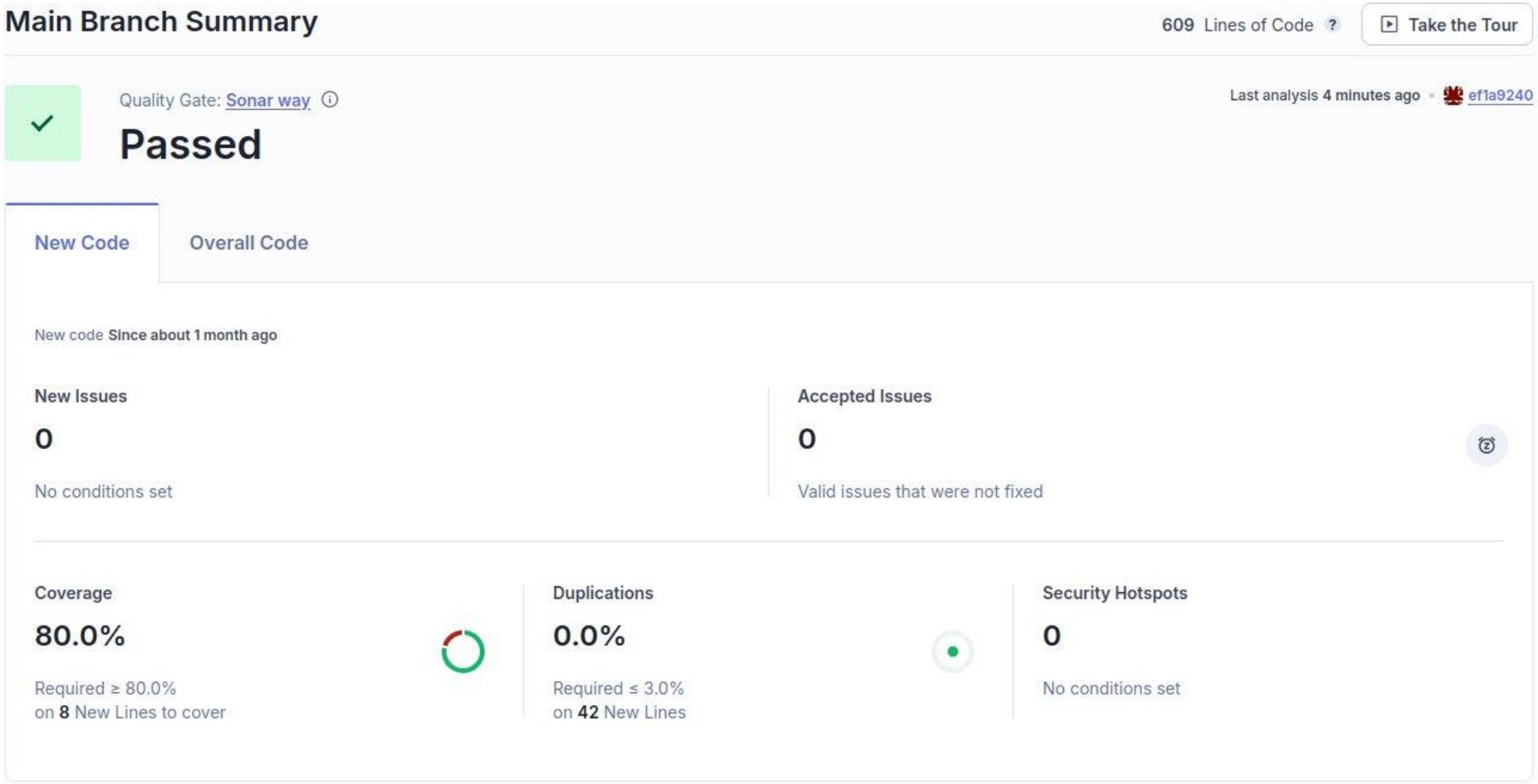


SonarCloud

En el *Project Dashboard* podemos ver el último análisis de la rama principal, las pull-requests y las ramas específicas

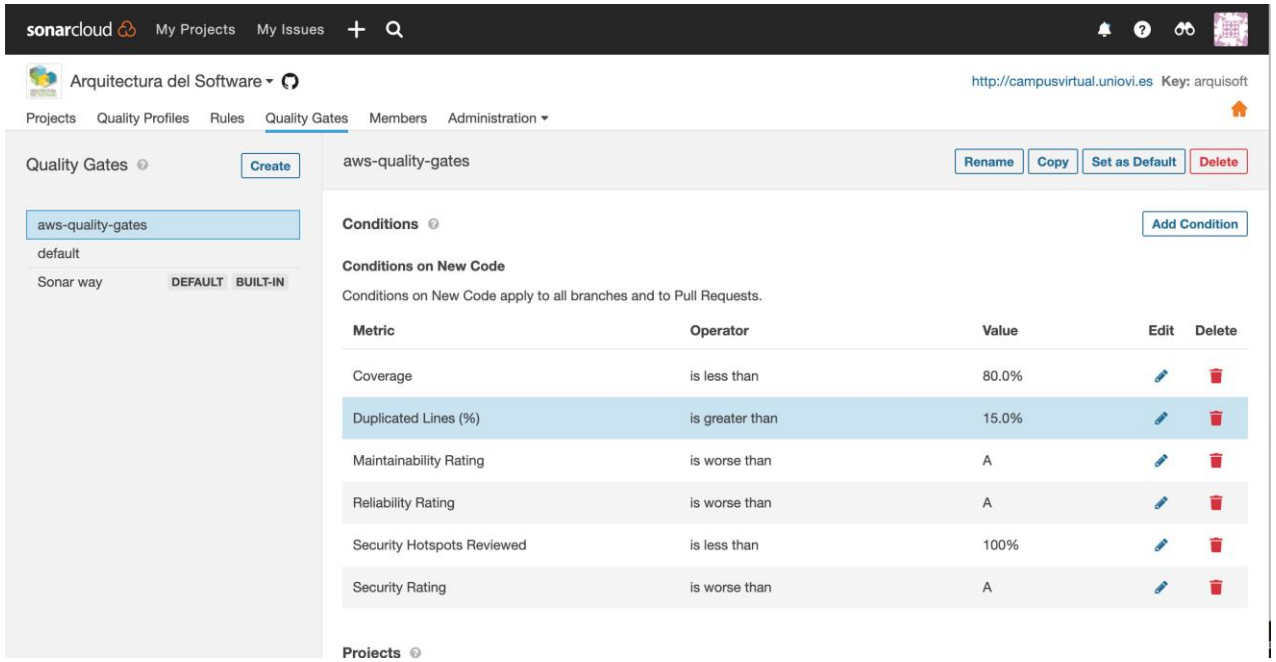


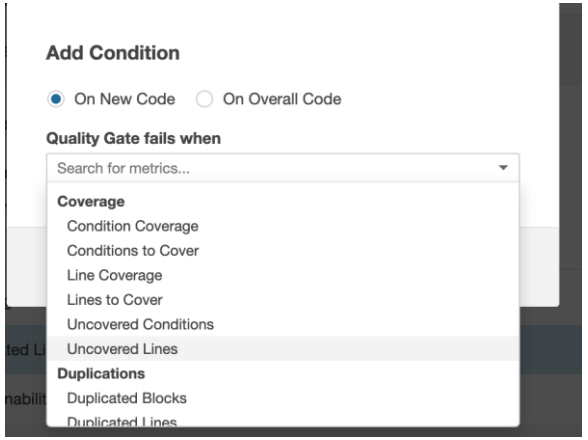
SonarCloud: Evolución de la calidad del proyecto



SonarCloud: Umbral de calidad

- En el nivel de la organización definimos distintos umbrales de calidad para asignarlos a los proyectos.





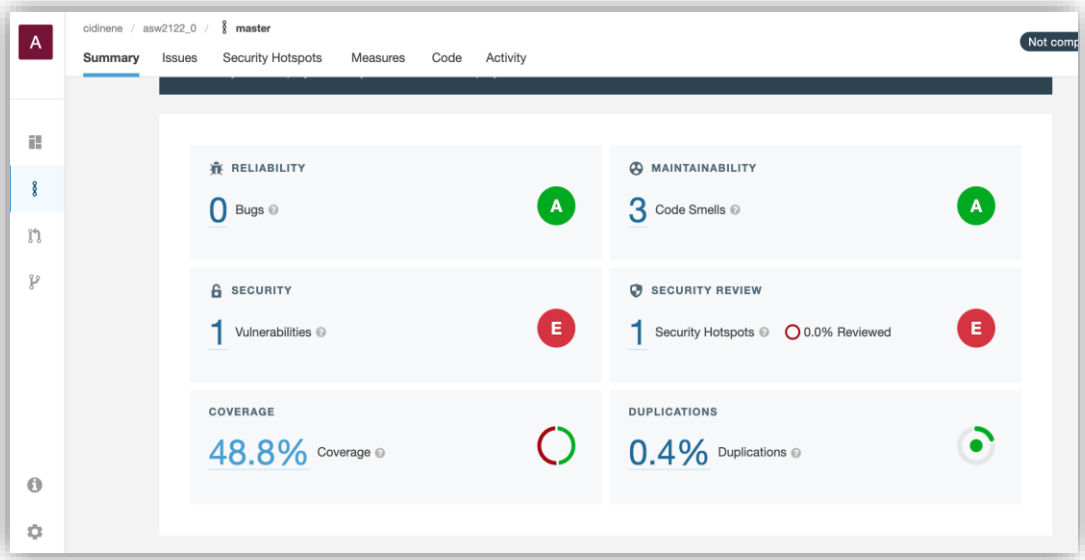
Ejemplo AWS-Quality-Gates , se puede incrementar el porcentaje de líneas duplicadas que se pueden encontrar antes de lanzar una excepción

SonarCloud: Umbral de calidad

Lo que se conoce como **Quality Gate** es la definición de condiciones que nuestro proyecto debe alcanzar.

- Estas condiciones requieren distintos aspectos: cobertura de código, análisis estático del código, líneas duplicadas, ..

yovi_o tiene configurada la calidad de código con SonarCloud.

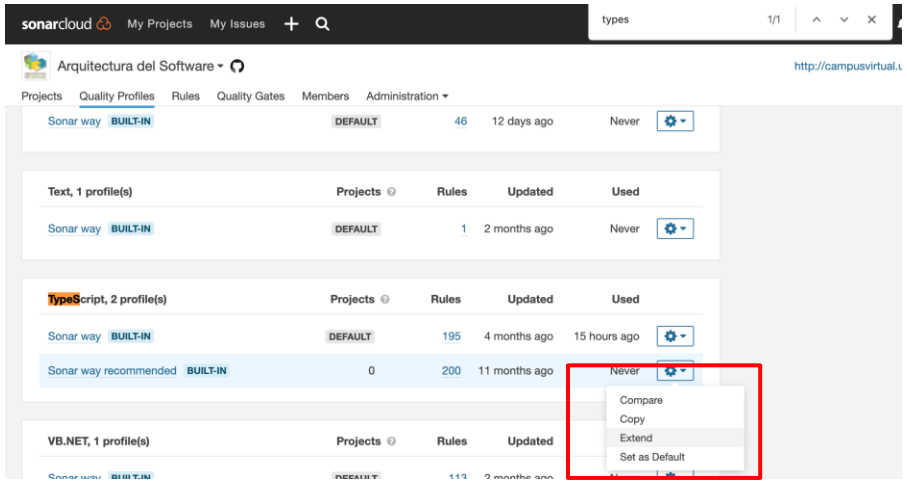


SonarCloud: Perfiles y reglas

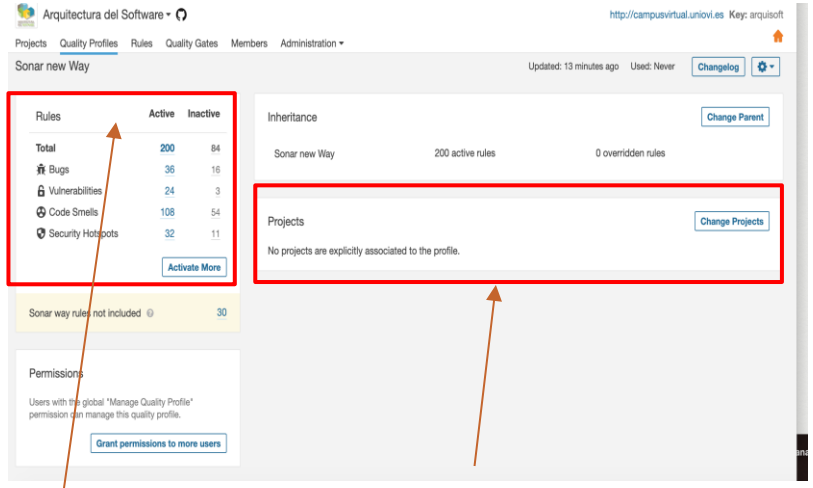
Las reglas están definidas en los perfiles

Podemos añadir, desactivar y actualizar reglas creando un nuevo perfil :

- Copiar un perfil padre – Cambiarlo – asociarlo al proyecto



Create a new profile



Set the profile rules

Associate the profile to the project

Configuración de reglas

sonarcloud.io/organizations/arquisoft/rules?qprofile=AX-mgR2YnzNFv0H6nzDH&activation=true

sonarcloud My Projects My Issues type 1/1 Paused

Arquitectura del Software http://campusvirtual.uniovi.es Key: arquisoft

Projects Quality Profiles Rules Quality Gates Members Administration

Filters Clear All Filters

Search for rules...

Language

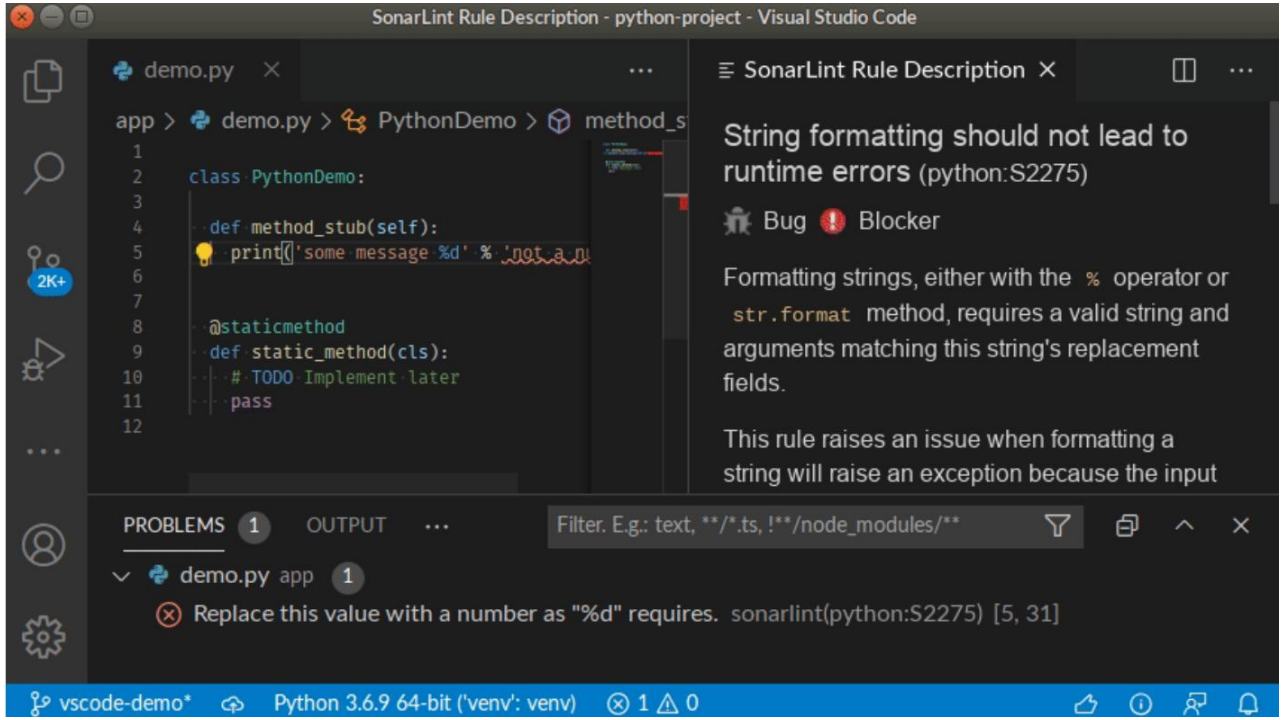
Type
- Bug 36
- Vulnerability 24
- Code Smell 108
- Security Hotspot 32

Tag
Repository
Default Severity
Status
Security Category
Available Since
Quality Profile SONAR N... Clear

Inheritance

Bulk Change ↑ ↓ to select rules ← → to navigate 1 / 200 rules

"===" and "!==" should be used instead of "==" and "!="	TypeScript	Code Smell	suspicious	Deactivate
"arguments.caller" and "arguments.callee" should not be used	TypeScript	Code Smell	obsolete	Deactivate
"await" should not be used redundantly	TypeScript	Code Smell	redundant	Deactivate
"await" should only be used with promises	TypeScript	Code Smell	confusing	Deactivate
"catch" clauses should do more than rethrow	TypeScript	Code Smell	clumsy, error-ha...	Deactivate
"default" clauses should be last	TypeScript	Code Smell		Deactivate
"delete" should be used only with object properties	TypeScript	Bug		Deactivate
"delete" should not be used on arrays	TypeScript	Code Smell		Deactivate
"for in" should not be used with iterables	TypeScript	Code Smell		Deactivate
"for of" should be used with Iterables	TypeScript	Code Smell	clumsy	Deactivate
"for" loop increment clauses should modify the loops' counters	TypeScript	Code Smell	confusing	Deactivate

Ver las alertas mientras programamos

- https://marketplace.visualstudio.com/items?itemName=SonarSource.sonarlint-vscode